Structures de données avancées

Informatique — Licence 2 (L2)

Cours INF-L2-11

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Choisir et implémenter les structures de données adaptées à chaque problème : complexité en notation grand O, listes chaînées, piles et files, arbres binaires et arbres de recherche, tables de hachage et graphes avec leurs parcours en largeur et en profondeur.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Complexité algorithmique : la notation grand O
• Leçon 2 — Listes chaînées simples et doublement chaînées
• Leçon 3 — Piles et files : implémentations et applications
• Leçon 4 — Arbres binaires et leurs parcours
• Leçon 5 — Arbres binaires de recherche et équilibrage : initiation
• Leçon 6 — Tables de hachage : principes et collisions
• Leçon 7 — Graphes : représentations et parcours en largeur et en profondeur
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as manipulé des variables, des tableaux et des listes. Mais un programme réel ne se contente pas de stocker des données : il doit les retrouver vite, les modifier souvent, les parcourir dans un ordre utile. Or, selon la façon dont les données sont organisées en mémoire, une même opération peut coûter une fraction de seconde ou devenir désespérément lente quand les données grandissent. Choisir la bonne organisation, c'est précisément l'objet des structures de données. Pense à un annuaire de cent mille étudiants : chercher un nom en lisant la liste du début à la fin passe inaperçu sur un carnet de vingt noms, mais devient insupportable à grande échelle. Toute la matière de ce cours tient dans cette question : comment organiser les données pour que les opérations qui comptent restent rapides ?

Ce cours présente les structures fondamentales de l'informatique : listes chaînées, piles, files, arbres, tables de hachage et graphes. Pour chacune, nous verrons la même démarche : comprendre l'idée, l'implémenter en Python, mesurer le coût de ses opérations avec la notation grand O, et reconnaître les situations où elle s'impose. Toutes les implémentations sont écrites à la main, sans raccourci, pour que le mécanisme devienne évident. Tu verras que ces structures ne sont pas des curiosités théoriques : la pile fait fonctionner le bouton « retour » de ton navigateur, la file ordonne les impressions d'un secrétariat, la table de hachage se cache derrière chaque dictionnaire Python, et les graphes décrivent aussi bien un plan de ville qu'un réseau d'amis. À la fin du cours, tu sauras non seulement les programmer, mais surtout décider laquelle employer — et expliquer pourquoi.

Un fil conducteur traverse le cours : la complexité. Avant même de coder une structure, demande-toi ce que coûteront ses opérations principales — accès, recherche, insertion, suppression — quand le nombre d'éléments n grandit. C'est ce réflexe, plus que la connaissance d'un langage, qui distingue un programmeur capable d'écrire un programme qui « passe à l'échelle » d'un programmeur dont le logiciel s'effondre dès que les données dépassent le millier d'éléments. Chaque leçon se termine par un exercice corrigé : refais-le sans regarder la solution, puis exécute le code pour confronter ton raisonnement à la machine — en matière de structures de données, c'est ce va-et-vient entre la main et le programme qui rend les ordres de visite et les coûts vraiment évidents.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Exprimer la complexité temporelle d'un algorithme simple avec la notation grand O.
- Implémenter une liste chaînée simple (insertion en tête, parcours, recherche) et expliquer ses coûts.
- Utiliser une pile et une file, et citer une application concrète de chacune.
- Construire un arbre binaire et produire ses parcours préfixe, infixe et postfixe.
- Insérer et rechercher dans un arbre binaire de recherche, et expliquer le rôle de la hauteur.
- Expliquer le principe d'une table de hachage, la notion de collision et le chaînage.
- Représenter un graphe par dictionnaire d'adjacence et le parcourir en largeur (BFS) et en profondeur (DFS).

Leçon 1 — Complexité algorithmique : la notation grand O

Pour comparer deux algorithmes, on ne chronomètre pas : le temps mesuré dépend de la machine, du langage et de la charge du système. On compte plutôt le nombre d'opérations élémentaires (comparaisons, affectations, accès mémoire) en fonction de la taille n des données. La notation grand O résume cette croissance en ne gardant que le terme dominant, sans constante : si un algorithme fait environ 3n + 5 opérations, on dit qu'il est en O(n), car quand n grandit, c'est n qui décide du coût.

Les ordres de croissance à connaître forment une échelle. O(1), constant : le coût ne dépend pas de n, comme l'accès à une case de tableau par son indice. O(log n), logarithmique : le problème est divisé par deux à chaque étape, comme la recherche dichotomique. O(n), linéaire : on touche chaque élément une fois. O(n log n) : celui des bons tris. O(n^2), quadratique : deux boucles imbriquées, qui explose vite — pour n = 1 000 000, n^2 vaut 10^{12}. Enfin O(2^n), exponentiel, devient inutilisable dès que n dépasse quelques dizaines.

La notation grand O décrit en général le pire cas, celui qui garantit une borne supérieure du coût. Elle sert à prédire le comportement à grande échelle : un algorithme en O(n^2) fonctionne très bien sur 100 éléments mais peut prendre des

heures sur un million. Dans l'exemple ci-dessous, on compte réellement les étapes d'une recherche linéaire et d'une recherche dichotomique dans le pire cas, sur un tableau de 1 024 éléments : la linéaire parcourt tout, la dichotomique n'a besoin que de 11 comparaisons, car $2^{10} = 1\,024$.

L'essentiel
La complexité grand O mesure comment le nombre d'opérations grandit avec la taille n des données, dans le pire cas. Échelle à retenir : $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.

Exemple — Compter les étapes : recherche linéaire contre dichotomie

```
def recherche_lineaire(t, x):
etapes = 0
for v in t:
etapes += 1
if v == x:
return etapes
return etapes

def recherche_dichotomique(t, x):
etapes = 0
g, d = 0, len(t) - 1
while g <= d:
etapes += 1
m = (g + d) // 2
if t[m] == x:
return etapes
elif t[m] < x:
g = m + 1
else:
d = m - 1
return etapes

t = list(range(1, 1025)) # 1024 elements tries
print(recherche_lineaire(t, 1024)) # pire cas : 1024
print(recherche_dichotomique(t, 1024)) # pire cas : 11
```

À retenir

- On compte les opérations en fonction de n ; on ne garde que le terme dominant, sans les constantes.
- Le grand O décrit le pire cas : c'est une garantie sur le coût maximal.
- Diviser le problème par deux à chaque étape donne un coût en O(log n), extrêmement faible même pour de grandes données.

Exercice

1) Quelle est la complexité d'un algorithme qui parcourt une liste de n éléments avec deux boucles imbriquées, chacune allant de 1 à n ? 2) Un tableau trié contient 1 000 éléments. Sans coder, donne une borne du nombre de comparaisons d'une recherche

dichotomique dans le pire cas. 3) Classe ces complexités de la plus efficace à la moins efficace : $O(n^2)$, $O(1)$, $O(n)$, $O(\log n)$.

✔ Corrigé
1) La boucle intérieure s'exécute n fois pour chaque tour de la boucle extérieure, soit $n \times n = n^2$ opérations : complexité $O(n^2)$. 2) Chaque comparaison divise l'intervalle de recherche par deux ; il faut au plus environ $\log_2(1\,000) \approx 10$ comparaisons ($2^{10} = 1\,024 \geq 1\,000$) : on reste très loin des 1 000 comparaisons de la recherche linéaire. 3) $O(1) < O(\log n) < O(n) < O(n^2)$: constante d'abord, puis logarithmique, linéaire, et quadratique en dernier.

Leçon 2 — Listes chaînées simples et doublement chaînées

Un tableau stocke ses éléments dans des cases contiguës en mémoire, ce qui rend l'accès par indice immédiat, mais les insertions au milieu coûteuses : il faut décaler tous les éléments suivants. La liste chaînée prend le parti inverse : chaque élément vit dans un nœud qui contient une valeur et un lien (une référence) vers le nœud suivant. Les nœuds sont dispersés en mémoire ; on ne connaît que le premier, la tête, et on avance de lien en lien.

Cette organisation a des coûts symétriques de ceux du tableau. Insérer en tête est immédiat, en $O(1)$: on crée un nœud et on le branche devant l'ancienne tête. Supprimer en tête est aussi en $O(1)$. En revanche, atteindre le k-ième élément impose de traverser k nœuds : l'accès et la recherche sont en $O(n)$ dans le pire cas. Insérer en fin de liste coûte également $O(n)$ si l'on ne mémorise pas le dernier nœud, puisqu'il faut d'abord parcourir toute la liste pour le trouver.

La liste doublement chaînée ajoute à chaque nœud un lien vers le nœud précédent. On peut alors parcourir la liste dans les deux sens et supprimer un nœud dont on connaît l'adresse sans repartir de la tête, toujours en suivant des liens en $O(1)$ une fois le nœud atteint. Le prix à payer : un lien supplémentaire par nœud, donc plus de mémoire, et des branchements plus délicats à écrire. Dans l'exemple, on implémente une liste simplement chaînée avec insertion en tête, affichage et recherche.

L'essentiel
Liste chaînée : des nœuds reliés par des références. Insertion et suppression en tête en $O(1)$; accès au k-ième élément et recherche en $O(n)$. La version doublement chaînée ajoute un lien « précédent » pour naviguer dans les deux sens.

Exemple — Implémenter une liste chaînée simple en Python

```
class Noeud:
def __init__(self, valeur):
self.valeur = valeur
self.suivant = None

class ListeChainee:
def __init__(self):
self.tete = None

def inserer_tete(self, valeur): # O(1)
n = Noeud(valeur)
n.suivant = self.tete
self.tete = n

def afficher(self):
res = []
courant = self.tete
while courant is not None:
res.append(str(courant.valeur))
courant = courant.suivant
return " -> ".join(res)

def rechercher(self, valeur): # O(n)
courant = self.tete
position = 0
while courant is not None:
if courant.valeur == valeur:
return position
courant = courant.suivant
position += 1
return -1

liste = ListeChainee()
for v in [10, 20, 30]:
liste.inserer_tete(v)
print(liste.afficher()) # 30 -> 20 -> 10
print(liste.rechercher(20)) # 1
print(liste.rechercher(99)) # -1 (absent)
```

À retenir

• Un nœud = une valeur + un lien vers le suivant ; la liste entière tient dans la référence « tête ».

• Insérer en tête : O(1). Accéder au k-ième élément ou rechercher une valeur : O(n), on suit les liens un à un.

• Doublement chaînée : chaque nœud connaît aussi son précédent ; parcours dans les deux sens, au prix d'un lien de plus par nœud.

Exercice

À partir de la classe de l'exemple, on a construit la liste 30 -> 20 -> 10. 1) Écris le code d'une méthode inserer_queue(valeur) qui ajoute un nœud à la fin de la liste, en parcourant les nœuds depuis la tête (cas particulier : la liste vide). 2) Après l'appel inserer_queue(40), que doit afficher la liste ? 3) Quelle est la complexité de cette

insertion en queue, et pourquoi ?

✔ Corrigé
1) def inserer_queue(self, valeur) : nouveau = Noeud(valeur) ; si self.tete est None, self.tete = nouveau et on s'arrête ; sinon on avance avec un pointeur courant tant que courant.suivant n'est pas None, puis on pose courant.suivant = nouveau. 2) La liste devient 30 -> 20 -> 10 -> 40 (résultat vérifié par exécution). 3) O(n) : il faut traverser tous les nœuds pour atteindre le dernier, faute d'un accès direct. Si les insertions en queue sont fréquentes, on mémorise aussi la queue de la liste : l'insertion redevient O(1).

Leçon 3 — Piles et files : implémentations et applications

La pile est une structure à accès restreint : on n'ajoute et on ne retire qu'à une seule extrémité, le sommet. Le dernier élément empilé est donc le premier dépilé : on parle de fonctionnement LIFO (Last In, First Out), comme une pile d'assiettes. Les opérations sont empiler (push), dépiler (pop) et consulter le sommet. Elles servent partout : historique de navigation d'un navigateur (le bouton retour dépile), fonction « annuler » d'un éditeur, et surtout la pile d'appels qui gère les fonctions en cours d'exécution et la récursivité.

La file suit la discipline inverse, FIFO (First In, First Out) : on ajoute en queue et on retire en tête, comme une file d'attente à un guichet. Le premier arrivé est le premier servi. Les files organisent le travail en attente : documents envoyés à une imprimante partagée, messages en attente de traitement sur un serveur, tâches planifiées. Point de vigilance en Python : retirer en tête d'une liste ordinaire avec pop(0) décale tous les éléments et coûte O(n) ; la classe deque du module collections le fait en O(1) aux deux extrémités.

Avec une liste Python, une pile s'implémente en quelques lignes : append pour empiler, pop sans indice pour dépiler, tous deux en O(1) amorti. L'exemple ci-dessous utilise une pile pour vérifier qu'une expression a ses parenthèses, crochets et accolades correctement appariés — à chaque fermant rencontré, le sommet de la pile doit être l'ouvrant correspondant. La seconde partie montre une file d'impression avec deque : le premier document arrivé est le premier imprimé.

L'essentiel
Pile = LIFO (dernier entré, premier sorti) ; file = FIFO (premier entré, premier sorti). En Python : liste pour la pile (append/pop en O(1)), deque de collections pour la file (append/popleft en O(1)).

Exemple — Pile : parenthèses équilibrées ; file : file d'impression

```
from collections import deque

def parentheses_equilibrees(expr):
pile = []
paires = {')': '(', ']': '[', '}': '{'}
for c in expr:
if c in "([{":
pile.append(c) # empiler l'ouvrant
elif c in ")]}":
if not pile or pile.pop() != paires[c]:
return False
return len(pile) == 0

print(parentheses_equilibrees("(a+b)*[c]")) # True
print(parentheses_equilibrees("(a+b]")) # False

file_imp = deque()
file_imp.append("rapport.pdf")
file_imp.append("photo.png")
file_imp.append("notes.txt")
print(file_imp.popleft()) # rapport.pdf (premier arrive)
print(list(file_imp)) # ['photo.png', 'notes.txt']
```

À retenir

• LIFO pour la pile, FIFO pour la file : toute la différence est dans l'extrémité de retrait.

• Applications de la pile : retour en arrière, annulation, évaluation d'expressions, pile d'appels.

• Applications de la file : files d'attente, impressions, traitements différés ; en Python, utiliser deque plutôt que pop(0) sur une liste.

Exercice

1) En utilisant uniquement une liste comme pile (append et pop), écris un programme qui inverse le mot « IAM » et affiche le résultat. 2) Explique en une phrase pourquoi une pile permet cette inversion. 3) On enfile dans une file les valeurs 5, 8, 13 dans cet ordre, puis on défile deux fois : quelles valeurs sortent, et dans quel ordre ?

✓ Corrigé
1) On empile chaque lettre : pile = [] ; pour c dans « IAM » : pile.append(c) ; puis on dépile en concaténant : resultat = « » ; tant que pile : resultat += pile.pop() ; on affiche resultat. Le programme affiche MAI (vérifié par exécution). 2) La pile ressort les éléments dans l'ordre inverse de leur entrée (LIFO), ce qui renverse exactement la séquence. 3) La file est FIFO : sortent 5 puis 8, dans l'ordre d'arrivée ; il reste 13 dans la file.

Leçon 4 — Arbres binaires et leurs parcours

Un arbre est une structure hiérarchique faite de nœuds reliés de parent à enfants, sans cycle. Le nœud de départ est la racine ; un nœud sans enfant est une feuille ; la

hauteur de l'arbre est le nombre de nœuds du plus long chemin de la racine à une feuille. Un arbre binaire impose au plus deux enfants par nœud, distingués en gauche et droite. Les arbres décrivent naturellement les hiérarchies : dossiers et fichiers, organigrammes, expressions arithmétiques, arbres de décision.

Parcourir un arbre, c'est visiter chaque nœud exactement une fois, dans un ordre défini. Les parcours en profondeur explorent un sous-arbre entièrement avant de passer au suivant, et se déclinent selon le moment où l'on traite la racine : préfixe (racine, gauche, droite), infixe (gauche, racine, droite), postfixe (gauche, droite, racine). Ces parcours s'écrivent très naturellement de façon récursive, car un arbre est lui-même défini récursivement : un nœud et deux sous-arbres.

L'autre grande famille est le parcours en largeur, niveau par niveau, qui utilise une file — on le retrouvera dans les graphes. Retiens les usages typiques : le parcours préfixe sert à copier un arbre ou à écrire une expression en notation préfixée ; le postfixe évalue un arbre d'expression (les opérandes avant l'opérateur) ; l'infixe, sur un arbre binaire de recherche, produit les valeurs triées — c'est le pont vers la leçon suivante. Tout parcours visite n nœuds : sa complexité est O(n).

L'essentiel
Arbre binaire : chaque nœud a au plus deux enfants (gauche, droite). Parcours en profondeur : préfixe = racine-gauche-droite, infixe = gauche-racine-droite, postfixe = gauche-droite-racine. Parcours en largeur : niveau par niveau, avec une file.

Exemple — Construire un arbre et produire ses trois parcours

```
class Arbre:
def __init__(self, valeur, gauche=None, droite=None):
self.valeur = valeur
self.gauche = gauche
self.droite = droite

def prefixe(n):
if n is None:
return []
return [n.valeur] + prefixe(n.gauche) + prefixe(n.droite)

def infixe(n):
if n is None:
return []
return infixe(n.gauche) + [n.valeur] + infixe(n.droite)

def postfixe(n):
if n is None:
return []
return postfixe(n.gauche) + postfixe(n.droite) + [n.valeur]

# 8
# / \
# 3 10
# / \ \
# 1 6 14
arbre = Arbre(8, Arbre(3, Arbre(1), Arbre(6)),
Arbre(10, None, Arbre(14)))
print(prefixe(arbre)) # [8, 3, 1, 6, 10, 14]
print(infixe(arbre)) # [1, 3, 6, 8, 10, 14]
print(postfixe(arbre)) # [1, 6, 3, 14, 10, 8]
```

À retenir

• Vocabulaire : racine, feuille, hauteur (plus long chemin racine → feuille, compté en nœuds).

• Les trois parcours en profondeur ne diffèrent que par la position de la racine : avant, entre, ou après les sous-arbres.

• Un parcours visite chaque nœud une fois : O(n). La récursivité épouse la définition même de l'arbre.

Exercice

Soit l'arbre suivant : racine 5 ; fils gauche 3 (fils gauche 1, fils droit 4) ; fils droit 9 (fils gauche 7, pas de fils droit). 1) Dessine l'arbre et donne sa hauteur. 2) Donne ses parcours préfixe, infixe et postfixe. 3) Quelles sont ses feuilles ?

✔ Corrigé
1) La hauteur est 3 (chemin 5 → 3 → 1, ou 5 → 3 → 4, ou 5 → 9 → 7). 2) Préfixe : [5, 3, 1, 4, 9, 7] ; infixe : [1, 3, 4, 5, 7, 9] ; postfixe : [1, 4, 3, 7, 9, 5] — résultats vérifiés par exécution du code de la leçon sur cet arbre. 3) Les feuilles sont 1, 4 et 7 : ce sont les nœuds sans aucun enfant.

Leçon 5 — Arbres binaires de recherche et équilibrage : initiation

Un arbre binaire de recherche (ABR) ajoute une règle d'ordre à l'arbre binaire : pour tout nœud, toutes les valeurs de son sous-arbre gauche sont plus petites que la sienne, et toutes celles du sous-arbre droit sont plus grandes. Cette propriété transforme la recherche : à chaque nœud, un seul test suffit pour éliminer la moitié de l'arbre restant. On descend à gauche si la valeur cherchée est plus petite, à droite si elle est plus grande, exactement comme la recherche dichotomique dans un tableau trié.

Le coût d'une recherche n'est donc pas lié au nombre total de nœuds, mais à la hauteur h de l'arbre : la recherche est en O(h), tout comme l'insertion (on descend jusqu'à trouver un emplacement libre). Conséquence immédiate du parcours vu à la leçon précédente : le parcours infixe d'un ABR produit les valeurs dans l'ordre croissant — c'est un excellent test de cohérence d'une implémentation, et l'exemple le vérifie par le code.

Le piège : la hauteur n'est pas garantie. Si l'on insère des valeurs déjà triées (1, 2, 3, 4, 5), chaque nouveau nœud devient le fils droit du précédent : l'arbre dégénère en une sorte de liste chaînée de hauteur 5 pour 5 nœuds, et la recherche redevient en O(n). Un arbre équilibré de même taille aurait une hauteur d'environ $\log_2(n)$. Les arbres auto-équilibrés (AVL, arbres rouge-noir) réorganisent l'arbre après chaque insertion pour empêcher cette dérive : ce sont eux qui équipent les bibliothèques standard, par exemple pour les dictionnaires ordonnés.

L'essentiel
ABR : à gauche les plus petits, à droite les plus grands. Recherche et insertion en O(h), où h est la hauteur : O(log n) si l'arbre est équilibré, O(n) s'il dégénère (insertions triées). Le parcours infixe d'un ABR est trié.

Exemple — Insérer et rechercher dans un arbre binaire de recherche

```
class Arbre:
def __init__(self, valeur, gauche=None, droite=None):
self.valeur = valeur
self.gauche = gauche
self.droite = droite

def infixe(n):
if n is None:
return []
return infixe(n.gauche) + [n.valeur] + infixe(n.droite)

class ABR:
def __init__(self):
self.racine = None

def inserer(self, valeur):
self.racine = self._inserer(self.racine, valeur)

def _inserer(self, n, valeur):
if n is None:
return Arbre(valeur)
if valeur < n.valeur:
n.gauche = self._inserer(n.gauche, valeur)
elif valeur > n.valeur:
n.droite = self._inserer(n.droite, valeur)
return n

def rechercher(self, valeur):
n = self.racine
while n is not None:
if valeur == n.valeur:
return True
n = n.gauche if valeur < n.valeur else n.droite
return False

abr = ABR()
for v in [8, 3, 10, 1, 6, 14]:
abr.inserer(v)
print(infixe(abr.racine)) # [1, 3, 6, 8, 10, 14] : trie !
print(abr.rechercher(6)) # True
print(abr.rechercher(7)) # False
```

À retenir

- Propriété ABR : sous-arbre gauche < nœud < sous-arbre droit, pour tout nœud.
- Recherche et insertion en O(h) : tout dépend de la hauteur, pas du nombre total de nœuds.
- Insérer des valeurs triées dégénère l'arbre en liste (h = n) ; les arbres équilibrés (AVL, rouge-noir) évitent cela.

Exercice

On insère successivement les valeurs 12, 4, 20, 2, 6 dans un ABR vide. 1) Décris l'arbre obtenu (racine, enfants de chaque nœud) et donne sa hauteur. 2) Donne son parcours infixe. 3) Pour rechercher la valeur 6, quels nœuds sont visités ?

✔ Corrigé
1) 12 devient la racine ; 4 va à gauche de 12 ; 20 à droite ; 2 à gauche de 4 ; 6 à droite de 4. La hauteur est 3 (chemin 12 → 4 → 2). 2) Le parcours infixe donne [2, 4, 6, 12, 20] : bien trié, comme attendu pour un ABR (vérifié par exécution). 3) On visite 12 (6 < 12, on descend à gauche), puis 4 (6 > 4, à droite), puis 6 : trouvé en 3 comparaisons, soit exactement la hauteur de l'arbre — c'est le O(h) en action.

Leçon 6 — Tables de hachage : principes et collisions

Comment retrouver un élément sans parcourir une liste ni descendre dans un arbre ? La table de hachage calcule directement l'adresse de rangement à partir de la clé, grâce à une fonction de hachage h. Pour une table de taille 10 et des clés entières, la plus simple est h(clé) = clé modulo 10. L'insertion et la recherche deviennent alors quasi immédiates : on calcule la case, on y va directement — en O(1) en moyenne, quel que soit le nombre d'éléments. C'est le principe des dictionnaires de Python.

Le problème est inévitable : deux clés différentes peuvent donner la même adresse. Ici, 13 et 23 tombent tous deux dans la case 3 : c'est une collision. La technique de résolution la plus lisible est le chaînage : chaque case contient une petite liste des éléments qui s'y projettent, et la recherche compare les clés de cette liste une à une. Tant que la fonction de hachage répartit bien les clés et que la table n'est pas trop remplie, ces listes restent courtes et le coût moyen reste proche de O(1).

La qualité d'une table dépend donc de deux choix : une fonction de hachage qui distribue les clés uniformément, et un taux de remplissage raisonnable — quand la table se remplit, on l'agrandit et on rehache les clés. Dans le pire cas (toutes les clés dans la même case), la recherche redevient linéaire, en O(n) : le hachage est un pari sur la régularité, gagné presque toujours en pratique. L'exemple implémente une table par chaînage et montre la collision de 13 et 23 en case 3.

L'essentiel
Table de hachage : l'adresse d'une clé est calculée par une fonction h (ex. clé % taille). Accès moyen en O(1). Deux clés à la même adresse = collision, résolue ici par chaînage (une liste par case). Pire cas O(n) si tout se concentre.

Exemple — Une table de hachage par chaînage, avec collision

```
TAILLE = 10
table = [[] for _ in range(TAILLE)]

def hachage(cle):
return cle % TAILLE

def inserer(cle, valeur):
table[hachage(cle)].append((cle, valeur))

def rechercher(cle):
for k, v in table[hachage(cle)]:
if k == cle:
return v
return None

inserer(13, "Awa")
inserer(23, "Moussa") # 23 % 10 = 3 : collision avec 13
inserer(7, "Fatou")
print(table[3]) # [(13, 'Awa'), (23, 'Moussa')]
print(rechercher(23)) # Moussa
print(rechercher(99)) # None (absent)
```

À retenir

- Le hachage transforme une clé en adresse : pas de parcours, un accès moyen en O(1).
- Collision : deux clés, une même case. Le chaînage range les éléments d'une case dans une petite liste.
- Une bonne fonction de hachage répartit les clés uniformément ; un taux de remplissage maîtrisé garde les listes courtes.

Exercice

On utilise la fonction h(clé) = clé % 10 et le chaînage. On insère, dans cet ordre, les clés 5, 15, 25, 8. 1) Que contient la case 5 ? Et la case 8 ? 2) Pour rechercher la clé 25, combien de comparaisons faut-il au maximum dans sa case ? 3) Pourquoi dit-on que le coût est O(1) « en moyenne » et pas toujours ?

✔ Corrigé
1) 5 % 10 = 5, 15 % 10 = 5, 25 % 10 = 5 : la case 5 contient [5, 15, 25] dans l'ordre d'insertion ; 8 % 10 = 8 : la case 8 contient [8] (vérifié par exécution). 2) La clé 25 est la troisième de sa case : au maximum 3 comparaisons. 3) Le coût dépend de la répartition : si la fonction concentre beaucoup de clés dans quelques cases, les listes chaînées s'allongent et la recherche y devient linéaire. En moyenne, avec une fonction régulière et une table assez grande, chaque case contient très peu d'éléments : le coût effectif reste voisin de O(1).

Leçon 7 — Graphes : représentations et parcours en largeur et en profondeur

Un graphe modélise des relations entre objets : des sommets (villes, personnes, pages web) reliés par des arêtes (routes, amitiés, liens). Contrairement à l'arbre, il n'y a ni racine ni hiérarchie, et les cycles sont permis. Deux représentations dominent : la matrice d'adjacence, un tableau sommet × sommet qui dit si deux sommets sont reliés — simple mais coûteuse en mémoire pour les graphes creux — et la liste (ou dictionnaire) d'adjacence, qui associe à chaque sommet la liste de ses voisins : c'est celle qu'on utilise en Python dans cette leçon.

Le parcours en largeur (BFS, Breadth-First Search) explore le graphe par couches : d'abord le sommet de départ, puis tous ses voisins, puis les voisins des voisins. Il s'implémente avec une file : on enfile les sommets découverts et on les traite dans l'ordre d'arrivée. Propriété précieuse : dans un graphe non pondéré, le BFS découvre chaque sommet par un plus court chemin (en nombre d'arêtes) depuis le départ. Il sert donc aux itinéraires, à la diffusion d'information de proche en proche, et au calcul de distances.

Le parcours en profondeur (DFS, Depth-First Search) s'enfonce au contraire le plus loin possible le long d'un chemin avant de revenir en arrière ; il s'implémente par récursion ou avec une pile. Il sert à explorer exhaustivement : détecter des cycles, tester si un graphe est connexe, résoudre un labyrinthe. Dans les deux cas, on marque les sommets déjà vus pour ne jamais boucler, et chaque sommet et chaque arête sont traités une fois : complexité O(S + A), pour S sommets et A arêtes. L'exemple exécute les deux parcours sur le même graphe : les ordres obtenus diffèrent, et c'est normal.

L'essentiel
Graphe = sommets + arêtes, représenté ici par un dictionnaire d'adjacence. BFS : exploration par couches avec une file, donne les plus courts chemins en nombre d'arêtes. DFS : exploration en profondeur avec récursion ou pile. Tous deux en O(S + A) avec marquage des sommets vus.

Exemple — Parcours BFS et DFS d'un graphe en dictionnaire d'adjacence

```
from collections import deque

graphe = {
'A': ['B', 'C'],
'B': ['A', 'D', 'E'],
'C': ['A', 'F'],
'D': ['B'],
'E': ['B', 'F'],
'F': ['C', 'E'],
}

def bfs(g, depart):
visites = []
vus = {depart}
f = deque([depart])
while f:
s = f.popleft()
visites.append(s)
for voisin in g[s]:
if voisin not in vus:
vus.add(voisin)
f.append(voisin)
return visites

def dfs(g, depart):
visites = []
vus = set()
def visiter(s):
vus.add(s)
visites.append(s)
for voisin in g[s]:
if voisin not in vus:
visiter(voisin)
visiter(depart)
return visites

print(bfs(graphe, 'A')) # ['A', 'B', 'C', 'D', 'E', 'F']
print(dfs(graphe, 'A')) # ['A', 'B', 'D', 'E', 'F', 'C']
```

À retenir

• Dictionnaire d'adjacence : à chaque sommet, la liste de ses voisins ; économe pour les graphes creux.

• BFS = file → couches successives → plus courts chemins en nombre d'arêtes dans un graphe non pondéré.

• DFS = pile ou récursion → on s'enfonce puis on revient en arrière ; marquer les sommets vus est indispensable à cause des cycles.

Exercice

On reprend le graphe de l'exemple. 1) Donne l'ordre de visite du BFS en partant de C. 2) Donne l'ordre de visite du DFS en partant de C (les voisins sont examinés dans l'ordre de la liste d'adjacence). 3) Quel est le plus court chemin de A à F, en nombre d'arêtes ? Justifie avec le BFS.

✔ Corrigé
1) BFS depuis C : ['C', 'A', 'F', 'B', 'E', 'D'] — C d'abord, puis ses voisins A et F, puis les voisins non vus de A (B), de F (E), enfin D voisin de B (vérifié par exécution). 2) DFS depuis C : ['C', 'A', 'B', 'D', 'E', 'F'] — on s'enfonce C → A → B → D, retour à B, puis E, puis F depuis E (vérifié par exécution). 3) Le plus court chemin de A à F compte 2 arêtes : A → C → F. Le BFS depuis A atteint F dès la deuxième couche (voisins des voisins), ce qui certifie qu'aucun chemin en 1 arête n'existe et que 2 est optimal.

Travaux dirigés (TD)

TD 1 — Choisir la bonne structure pour chaque besoin

Pour chacun des quatre besoins ci-dessous, choisis la structure la plus adaptée parmi : tableau/liste ordinaire, liste chaînée, pile, file, arbre binaire de recherche, table de hachage. Justifie chaque choix par les opérations dominantes et leur complexité. a) Un annuaire qui, à partir d'un numéro matricule, affiche immédiatement le nom d'un étudiant, avec des consultations très fréquentes et peu d'insertions. b) Le bouton « page précédente » d'un navigateur, qui doit revenir en arrière étape par étape. c) Le guichet d'une administration qui sert les usagers dans l'ordre exact de leur arrivée. d) Un carnet de notes que l'on consulte toujours par ordre alphabétique croissant des noms, et dans lequel on insère souvent de nouveaux noms.

✔ Corrigé du TD 1
a) Table de hachage : l'opération dominante est la recherche par clé ; le hachage la fait en O(1) en moyenne, contre O(n) par balayage d'une liste. b) Pile : l'historique se consulte en sens inverse de l'enregistrement, exactement le LIFO — chaque retour dépile la page courante. c) File : l'équité impose le FIFO ; une deque donne enfiler et défiler en O(1). d) Arbre binaire de recherche (équilibré) : le parcours infixe produit l'ordre alphabétique à la demande, et insertion comme recherche restent en O(log n) ; une liste triée rendrait chaque insertion coûteuse (décalages), et un tableau non trié imposerait un tri complet à chaque affichage. Le point commun des quatre réponses : on choisit la structure d'après les opérations les plus fréquentes, pas d'après l'habitude.

TD 2 — Lire un arbre et un graphe : parcours et distances

On considère l'arbre binaire suivant : racine 20 ; fils gauche 10 (fils gauche 5, fils droit 15) ; fils droit 30 (fils gauche 25, fils droit 35). Et le graphe de la leçon 7 (sommets A à F, dictionnaire d'adjacence donné dans l'exemple). 1) Donne les parcours préfixe, infixe et postfixe de l'arbre, son nombre de feuilles et sa hauteur. 2) Cet arbre est-il un arbre binaire de recherche ? Justifie en deux manières différentes. 3) Pour le graphe, donne la distance (en arêtes) de A vers chaque autre sommet, en t'appuyant sur le BFS.

✔ Corrigé du TD 2
1) Préfixe : [20, 10, 5, 15, 30, 25, 35] ; infixe : [5, 10, 15, 20, 25, 30, 35] ; postfixe : [5, 15, 10, 25, 35, 30, 20]. Feuilles : 5, 15, 25, 35, donc 4 feuilles. Hauteur : 3. (Résultats vérifiés par exécution.) 2) Oui. Première justification : à chaque nœud, les valeurs de gauche sont inférieures et celles de droite supérieures (5, 15 encadrent 10 ; 25, 35 encadrent 30 ; 10 et 30 encadrent 20). Deuxième justification : le parcours infixe est strictement croissant, propriété caractéristique des ABR. 3) Distances depuis A : A = 0 ; B = 1 et C = 1 (voisins directs, première couche du BFS) ; D = 2, E = 2, F = 2 (atteints à la deuxième couche : A → B → D, A → B → E, A → C → F). Le BFS garantit que ces distances sont les plus courtes possibles en nombre d'arêtes.

Travaux pratiques (TP)

TP 1 — File d'attente d'un secrétariat : pile d'annulation et file de service

Le secrétariat d'un département reçoit des étudiants pour des demandes d'attestations. Tu vas simuler le service avec Python. 1) Crée une file (avec collections.deque) et enfile les arrivées dans cet ordre : « Awa », « Moussa », « Fatou », « Ibrahim ». 2) Affiche le prochain étudiant servi sans le retirer (regarder la tête de la file). 3) Sers deux étudiants (défile et affiche leur nom, dans l'ordre de service). 4) Un nouvel étudiant, « Mariam », arrive : enfile-la, puis affiche la file complète restante, dans l'ordre de service. 5) Bonus : le secrétariat veut pouvoir « rappeler » le dernier étudiant servi (par exemple si son dossier est incomplet). Explique quelle structure ajouter et pourquoi elle convient.

✔ Attendus du TP 1
Attendus (vérifiés par exécution du même type de code qu'à la leçon 3) : 1) la file contient, de la tête vers la queue : Awa, Moussa, Fatou, Ibrahim. 2) Le prochain servi est Awa (tête de file, accès par file[0] sans retrait). 3) Les deux servis, dans l'ordre : Awa puis Moussa — le FIFO garantit le respect de l'ordre d'arrivée. 4) Après l'arrivée de Mariam, la file restante est : Fatou, Ibrahim, Mariam. 5) Il faut une pile des étudiants déjà servis : chaque service empile le nom servi ; « rappeler » dépile alors exactement le dernier servi, en LIFO. La leçon du TP : file et pile se combinent — la file gère l'attente (équité envers ceux qui arrivent), la pile gère l'historique (retour sur ce qui vient de se passer).

Exercices corrigés supplémentaires

Exercice 1

Écris une fonction nb_feuilles(arbre) qui compte les feuilles (nœuds sans enfant) d'un arbre binaire représenté comme dans la leçon 4. Applique-la à l'arbre de l'exemple de la leçon 4 (racine 8 ; 3 avec enfants 1 et 6 ; 10 avec enfant droit 14) et donne le résultat.

✔ Corrigé
def nb_feuilles(n) : si n est None, retourner 0 ; si n.gauche est None et n.droite est None, retourner 1 ; sinon retourner nb_feuilles(n.gauche) + nb_feuilles(n.droite). Le cas de base « arbre vide » vaut 0, le cas « feuille » vaut 1, et tout autre nœud additionne les feuilles de ses deux sous-arbres. Sur l'arbre de la leçon 4, les feuilles sont 1, 6 et 14 : la fonction renvoie 3 (vérifié par exécution).

Exercice 2

Écris une fonction est_palindrome(mot) qui utilise une pile pour décider si un mot se lit de la même façon dans les deux sens. Teste-la sur « radar » et sur « iam », et donne les résultats.

✔ Corrigé
Principe : on empile toutes les lettres du mot, puis on dépile en reconstruisant une chaîne : la pile la restitue dans l'ordre inverse. def est_palindrome(mot) : pile = list(mot) ; inverse = « » ; tant que pile : inverse += pile.pop() ; retourner inverse == mot. Résultats vérifiés par exécution : « radar » donne True (son inverse est lui-même) ; « iam » donne False (son inverse est « mai »). Complexité : O(n) en temps, la pile stocke les n lettres.

Exercice 3

En réutilisant le code du BFS de la leçon 7, écris une fonction distances(graphe, depart) qui renvoie un dictionnaire associant à chaque sommet sa distance en nombre d'arêtes depuis le départ. Applique-la au graphe de la leçon 7 depuis le sommet A et donne le dictionnaire obtenu.

✔ Corrigé
On adapte le BFS : on initialise dist = {depart : 0} et une file contenant le départ ; à chaque sommet défilé s, pour chaque voisin v non encore dans dist, on pose dist[v] = dist[s] + 1 et on enfile v. Le marquage se fait au moment de l'enfilage, ce qui garantit que la première distance attribuée est la plus courte. Résultat vérifié par exécution depuis A : {'A': 0, 'B': 1, 'C': 1, 'D': 2, 'E': 2, 'F': 2} — cohérent avec le TD 2.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Accéder au k-ième élément d'une liste chaînée simple coûte, dans le pire cas…

A) O(1), par un accès direct

B) O(n), car il faut suivre les liens depuis la tête

C) O(log n), par dichotomie

2. Insérer un élément en tête d'une liste chaînée coûte…

A) O(n)

B) O(n²)

C) O(1)

3. Une pile fonctionne selon le principe…

A) LIFO : le dernier entré est le premier sorti

B) FIFO : le premier entré est le premier sorti

C) Accès libre à n'importe quel élément

4. Le parcours infixe d'un arbre binaire de recherche produit les valeurs…

A) Dans leur ordre d'insertion

B) Par ordre croissant

C) Niveau par niveau

5. La recherche dans un arbre binaire de recherche équilibré coûte…

A) O(n)

B) O(n²)

C) O(log n)

6. Le pire cas de recherche dans un ABR s'obtient quand…

A) L'arbre est dégénéré (insertions triées) : coût O(n)

B) L'arbre est complet : coût O(1)

C) L'arbre est vide : coût O(log n)

7. Une collision dans une table de hachage signifie que…

A) La table est pleine

B) Deux clés différentes produisent la même adresse

C) La clé est trop grande pour la table

8. Le parcours en largeur (BFS) d'un graphe s'implémente avec…

A) Une pile

B) Une table de hachage

C) Une file

9. Le parcours en profondeur (DFS) s'implémente naturellement avec…

A) Une file

B) La récursion (ou une pile)

C) Une matrice d'adjacence uniquement

10. Parcourir entièrement un arbre de n nœuds coûte…

A) O(n) : chaque nœud est visité une fois

B) O(log n)

C) O(n²)

Corrigés du QCM

1. Réponse B — Sans indice ni accès direct, on traverse les nœuds un à un depuis la tête : O(n) au pire.

2. Réponse C — On crée un nœud et on le branche devant l'ancienne tête : un nombre fixe d'opérations, O(1).

3. Réponse A — La pile ne donne accès qu'à son sommet : le dernier empilé est le premier dépilé (LIFO).

4. Réponse B — Infixe = gauche, racine, droite ; avec la propriété ABR, cela donne exactement la suite triée.

5. Réponse C — La hauteur d'un arbre équilibré est en O(log n), et la recherche suit un seul chemin de cette longueur.

6. Réponse A — Insérer des valeurs triées crée une chaîne de fils droits : hauteur n, recherche en O(n).

7. Réponse B — La fonction de hachage envoie deux clés distinctes vers la même case ; le chaînage permet de les y ranger toutes les deux.

8. Réponse C — La file traite les sommets dans leur ordre de découverte, ce qui produit l'exploration par couches du BFS.

9. Réponse B — DFS s'enfonce le long d'un chemin : la récursion, ou explicitement une pile, gère les retours en arrière.

10. Réponse A — Préfixe, infixe, postfixe ou largeur : tout parcours visite chaque nœud exactement une fois, donc O(n).

Résumé — les notions importantes

• La notation grand O exprime la croissance du coût en fonction de n, dans le pire cas : O(1), O(log n), O(n), O(n log n), O(n^2) forment l'échelle de référence.

• Liste chaînée : nœuds reliés par des références ; insertion en tête en O(1), accès et recherche en O(n) ; la version doublement chaînée navigue dans les deux sens.

• Pile = LIFO, file = FIFO : deux disciplines d'accès qui couvrent l'essentiel des files d'attente et des mécanismes d'annulation ; en Python, liste pour la pile, deque pour la file.

• Arbre binaire : parcours préfixe, infixe, postfixe (en profondeur, récursifs) et en largeur (par couches, avec une file), chacun en O(n).

• Arbre binaire de recherche : recherche et insertion en O(h) ; O(log n) si l'arbre est équilibré, O(n) s'il dégénère ; son parcours infixe est trié.

• Table de hachage : adresse calculée à partir de la clé, accès moyen en O(1) ; les collisions se résolvent notamment par chaînage.

• Graphe : représenté par dictionnaire d'adjacence ; BFS (file) pour les couches et les plus courts chemins en arêtes, DFS (pile/récursion) pour l'exploration en profondeur, en O(S + A).

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Complexité algorithmique : la notation grand O
On compte les opérations en fonction de n ; on ne garde que le terme dominant, sans les constantes. ; Le grand O décrit le pire cas : c'est une garantie sur le coût maximal. ; Diviser le problème par deux à chaque étape donne un coût en O(log n), extrêmement faible même pour de grandes données.

Fiche 2 — Listes chaînées simples et doublement chaînées
Un nœud = une valeur + un lien vers le suivant ; la liste entière tient dans la référence « tête ». ; Insérer en tête : O(1). Accéder au k-ième élément ou rechercher une valeur : O(n), on suit les liens un à un. ; Doublement chaînée : chaque nœud connaît aussi son précédent ; parcours dans les deux sens, au prix d'un lien de plus par nœud.

Fiche 3 — Piles et files : implémentations et applications
LIFO pour la pile, FIFO pour la file : toute la différence est dans l'extrémité de retrait. ; Applications de la pile : retour en arrière, annulation, évaluation d'expressions, pile d'appels. ; Applications de la file : files d'attente, impressions, traitements différés ; en Python, utiliser deque plutôt que pop(0) sur une liste.

Fiche 4 — Arbres binaires et leurs parcours
Vocabulaire : racine, feuille, hauteur (plus long chemin racine → feuille, compté en nœuds). ; Les trois parcours en profondeur ne diffèrent que par la position de la racine : avant, entre, ou après les sous-arbres. ; Un parcours visite chaque nœud une fois : O(n). La récursivité épouse la définition même de l'arbre.

Fiche 5 — Arbres binaires de recherche et équilibrage : initiation
Propriété ABR : sous-arbre gauche < nœud < sous-arbre droit, pour tout nœud. ; Recherche et insertion en O(h) : tout dépend de la hauteur, pas du nombre total de nœuds. ; Insérer des valeurs triées dégénère l'arbre en liste (h = n) ; les arbres équilibrés (AVL, rouge-noir) évitent cela.

Fiche 6 — Tables de hachage : principes et collisions
Le hachage transforme une clé en adresse : pas de parcours, un accès moyen en O(1). ; Collision : deux clés, une même case. Le chaînage range les éléments d'une case dans une petite liste. ; Une bonne fonction de hachage répartit les clés uniformément ; un taux de remplissage maîtrisé garde les listes courtes.

Fiche 7 — Graphes : représentations et parcours en largeur et en profondeur
Dictionnaire d'adjacence : à chaque sommet, la liste de ses voisins ; économe pour les graphes creux. ; BFS = file → couches successives → plus courts chemins en nombre d'arêtes dans un graphe non pondéré. ; DFS = pile ou récursion → on s'enfonce puis on revient en arrière ; marquer les sommets vus est indispensable à cause des cycles.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Algorithme 22: Complexité d'un algorithme O(1), O(log n), O(n), O(n log n), O(2^n), O(n!).

Chaîne : Hassan EL BAHI · Langue : Français · Niveau : Intermédiaire

Un cours complet sur la complexité : définition, notation grand O et représentation graphique de chaque famille, des exemples concrets et des exercices corrigés pour chaque ordre de croissance. Le prolongement direct de la leçon 1.

https://www.youtube.com/watch?v=IHdf13wRhO0

Leçon 2 — Maîtriser les Listes Chaînées

Chaîne : Mon Prof IA · Langue : Français · Niveau : Intermédiaire

Les listes chaînées en profondeur : cellules et maillons, attributs valeur et suivant, calcul de la longueur et accès au n-ième élément en Python. Idéal pour revoir l'implémentation de la leçon 2 en vidéo.

https://www.youtube.com/watch?v=Jkku6ELRRS4

Leçon 3 — Python Avancé (1/3) | Leçon 5 : La Boîte à Outils du Spécialiste - Maîtriser le Module collections

Chaîne : TeachSmart AI Academy · Langue : Français · Niveau : Intermédiaire

Le module collections de Python passé au crible, avec la deque au premier plan : l'outil qui implémente files et piles avec des ajouts et retraits en O(1) aux deux extrémités, exactement ce qu'utilise la leçon 3.

https://www.youtube.com/watch?v=Iw1sjOiT8-s

Leçon 4 — Arbres Binaires pour Debutants - Cours Complet en Francais

Chaîne : lioncode · Langue : Français · Niveau : Débutant

Les arbres binaires de A à Z : terminologie, types d'arbres, arbre binaire de recherche, insertions et parcours en profondeur et en largeur, jusqu'aux arbres AVL. Couvre les leçons 4 et 5 avec des visualisations claires.

https://www.youtube.com/watch?v=w2dukxTDzSs

Leçon 7 — 10 Maîtriser les Graphes

Chaîne : Elisabeth Le Prettre · Langue : Français · Niveau : Intermédiaire

Une révision complète des graphes : vocabulaire, représentations par matrice et listes d'adjacence avec leurs complexités, puis les parcours BFS et DFS et leurs applications comme les plus courts chemins. Parfait pour consolider la leçon 7.

https://www.youtube.com/watch?v=ZwY-jk6HTqc

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Cormen, Leiserson, Rivest & Stein — « Introduction à l'algorithmique » (Dunod) : les chapitres sur les structures de données élémentaires, les arbres de recherche et les tables de hachage font référence.

• Documentation Python (docs.python.org/fr) — notamment le module collections et la classe deque, utilisés dans ce cours.

• Downey, A. — « Think Data Structures » (O'Reilly, accès libre en anglais) : une approche par la pratique des structures et de leur complexité.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.